

Backtracking: Parte II

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

Repaso de la Clase Anterior



Cottonbro Studio

Repaso

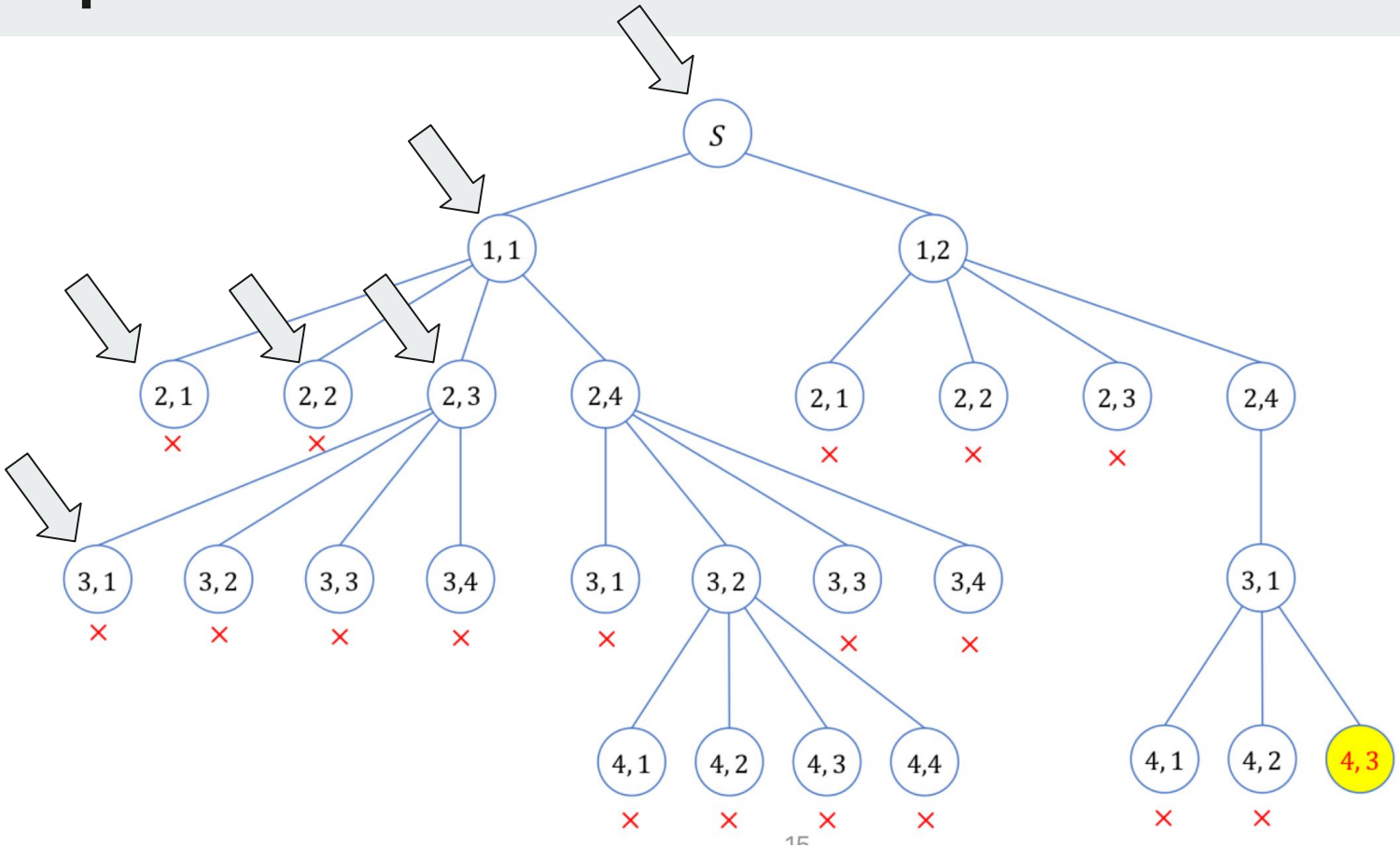
Anteriormente, en “Teoría de Algoritmos”...

- Fuerza Bruta y Backtracking
- Problema de las N Reinas
- Independent Set
- Juegos
- Camino Hamiltoniano
- Coloreo de Grafos
- Casco Convexo



Cap 2

Repaso: Árbol de Decisiones

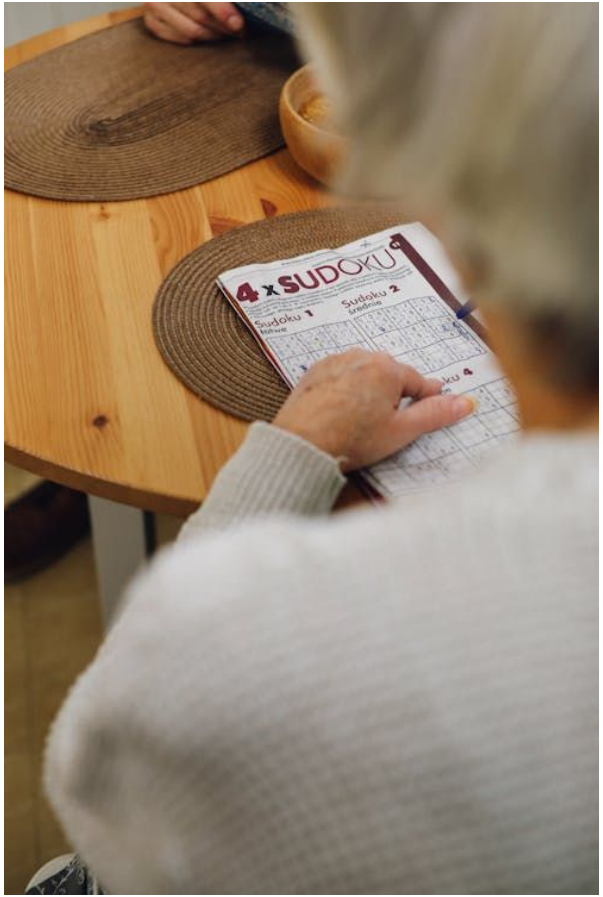


Repaso: Funcionamiento de Backtracking

Recorre recursivamente el Árbol de Decisiones usando DFS:

1. Verifica el candidato parcial:
    a. Si es la solución:
        i. la devuelve
2. Incrementa el candidato parcial
3. Si no es válido:
    a. retrocede y vuelve a 2
4. Si es válido:
    a. recorre recursivamente desde 1 con el candidato parcial actual
5. Llegado a este punto, el problema no tiene solución

Sudoku



T. Leish

Sudoku: Definición

- Se tiene una grilla de 9x9 celdas
- La grilla está organizada en 9 bloques de 3x3 celdas
- En cada bloque deben colocarse todos los dígitos del 1 al 9, sin repetirlos
- Cada columna y cada fila de la grilla debe tener todos los dígitos del 1 al 9, sin repetirlos
- Algunas de las celdas ya tienen números asignados

Se desea encontrar una asignación de números a grillas que cumpla con todas las condiciones antes mencionadas

Sudoku: Análisis

¿Cómo lo podríamos resolver?

¿Cómo se resolvería por Fuerza Bruta?

¿Cómo se resolvería por Backtracking?

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Tim Stellmach

Sudoku: Análisis (cont.)

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Colin M.L. Burnett

Sudoku: Algoritmo

```
def sudoku(actual, M):
  siguiente=proxima_posicion(actual, M)
  if siguiente >= 81: return True
  for num in range (1,10):
    if puedo_poner(num, siguiente, M):
      M[fila(siguiente)][columna(siguiente)] = num
      if sudoku(siguiente, M): return True
  M[fila(siguiente)][columna(siguiente)]=0
  return False
```

Verificar candidato parcial

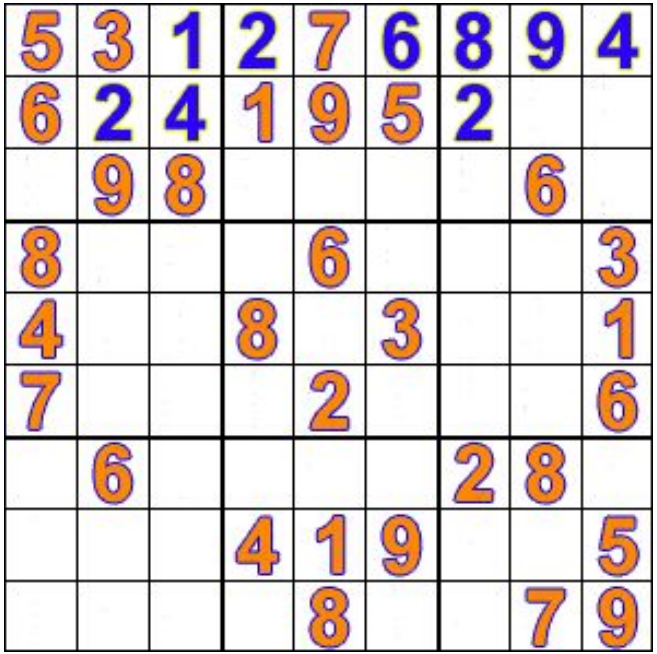
Incrementar candidato parcial

Si no es válido, prueba con otro

Si es válido, llama recursivamente

Backtrack

Sudoku: Resolviendo con Backtraking



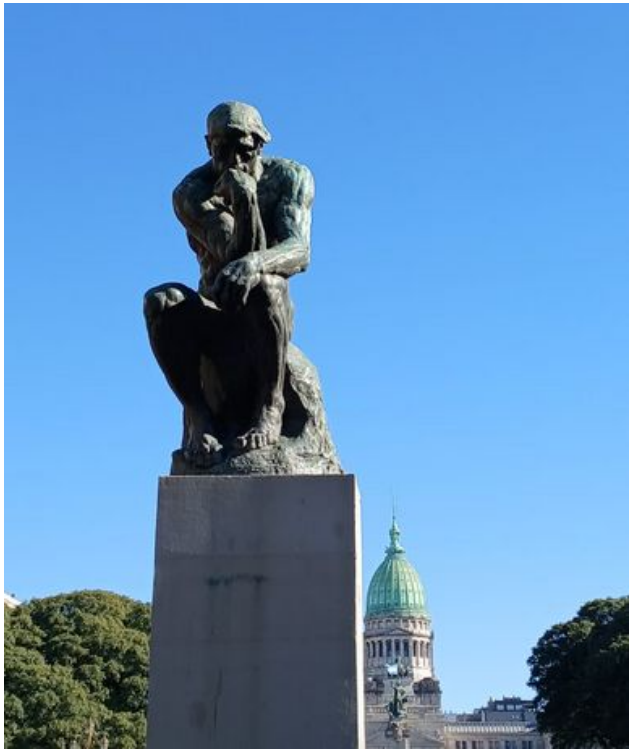
Simpsons contributor

Sudoku: Momento Filosófico

¿Se puede establecer una analogía entre este problema y alguno de grafos que ya conozcamos?

¿Qué serían los vértices?

¿Qué serían las aristas?

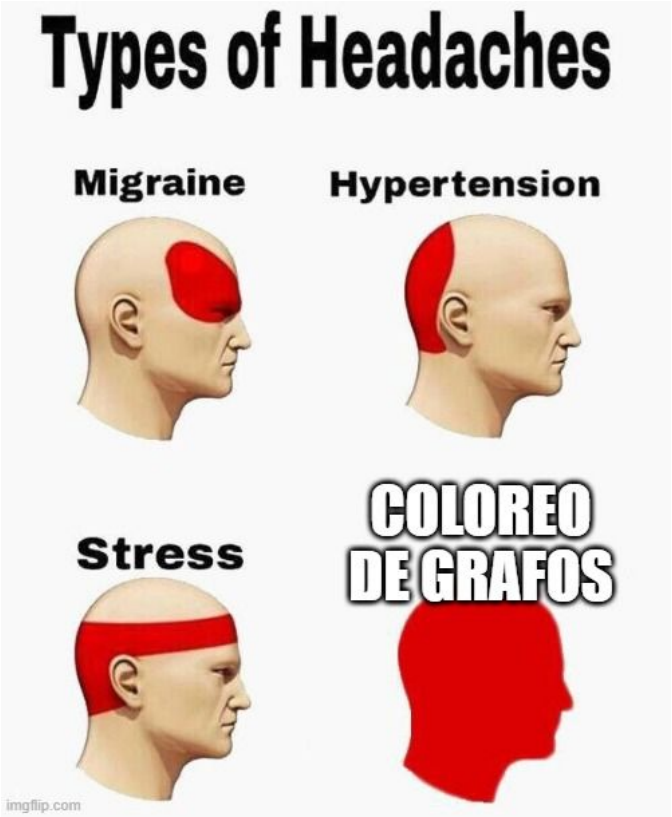


Sudoku: Momento Filosófico (cont.)

- Los vértices representan cada celda del Sudoku
- Las aristas unen vértices que no pueden tener el mismo dígito:
  - Vértices de la misma fila
  - Vértices de la misma columna
  - Vértices del mismo bloque

¿Se parece a algo que hayamos visto?

Sudoku: Momento Filosófico (cont.)



Problema del Caballo de Ajedrez (Knight's Tour)



AmazingArtWork

Problema del Caballo: Definición

- Se tiene un tablero de ajedrez de 8x8 casilleros
- Se tiene una pieza del caballo
- El caballo se mueve en forma de L: dos casilleros en una dirección y luego uno en dirección perpendicular (2x1)

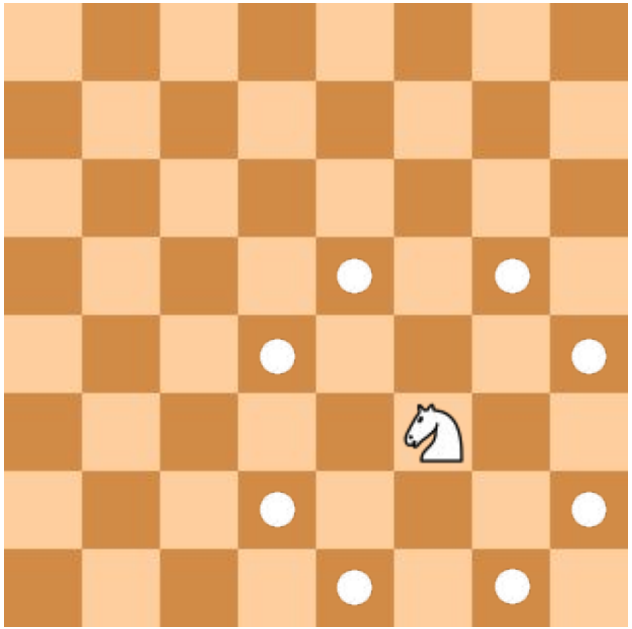
Determinar una secuencia de movimientos que permita al caballo pasar por todos los casilleros del tablero sin repetirlos

Problema del Caballo: Análisis

¿Cómo lo podríamos resolver?

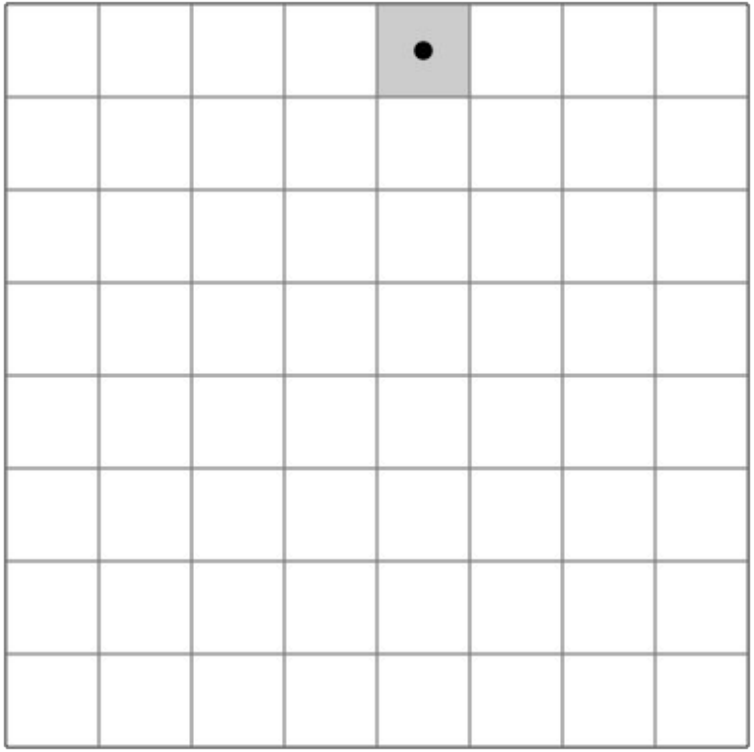
¿Cómo se resolvería por Fuerza Bruta?

¿Cómo se resolvería por Backtracking?



Posibles movimientos del Caballo de Ajedrez

Problema del Caballo: Análisis (cont.)



Ilmari Karonen

Problema del Caballo: Pseudocódigo

```
def caballo(paso):
  if completo(): return True
  x, y = obtener_posicion_actual_caballo()
  for fila, col in movimientos_caballo(x,y):
    if not dentro_de_tablero(fila, col): continue
    if casillero_ya_marcado(fila, col): continue
    mover_a_posicion(fila, col, paso)
    if (caballo(paso+1)):
      return True
    volver_a_posicion(x,y)
  return False
```

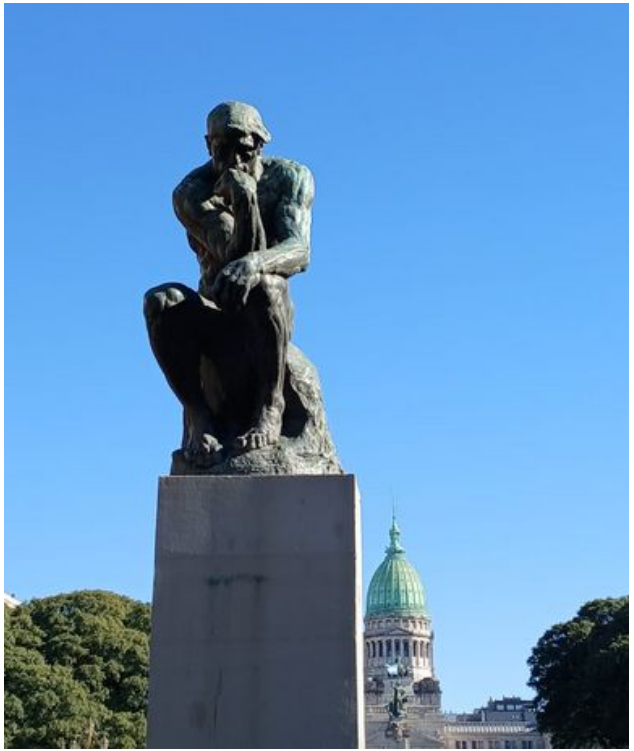
Créditos: autor desconocido

Problema del Caballo: Momento Filosófico

¿Se puede establecer una analogía entre este problema y alguno de grafos que ya conozcamos?

¿Qué serían los vértices?

¿Qué serían las aristas?

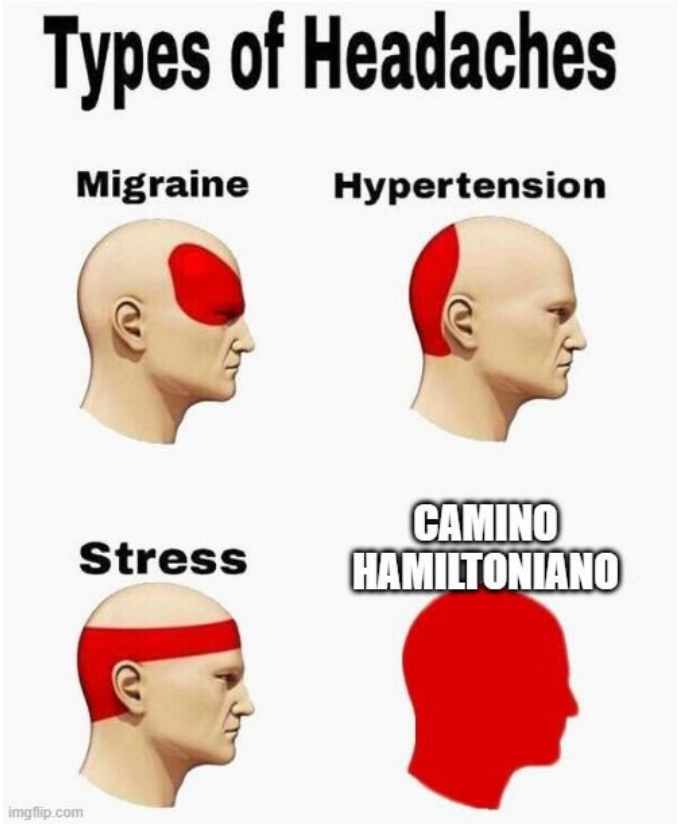


Problema del Caballo: Momento Filosófico (cont.)

- Los vértices representan cada casillero del tablero
- Las aristas unen un vértice con los vértices a los que podría moverse el caballo:
  - Movimiento en forma de L: 2x1

¿Se parece a algo que hayamos visto?

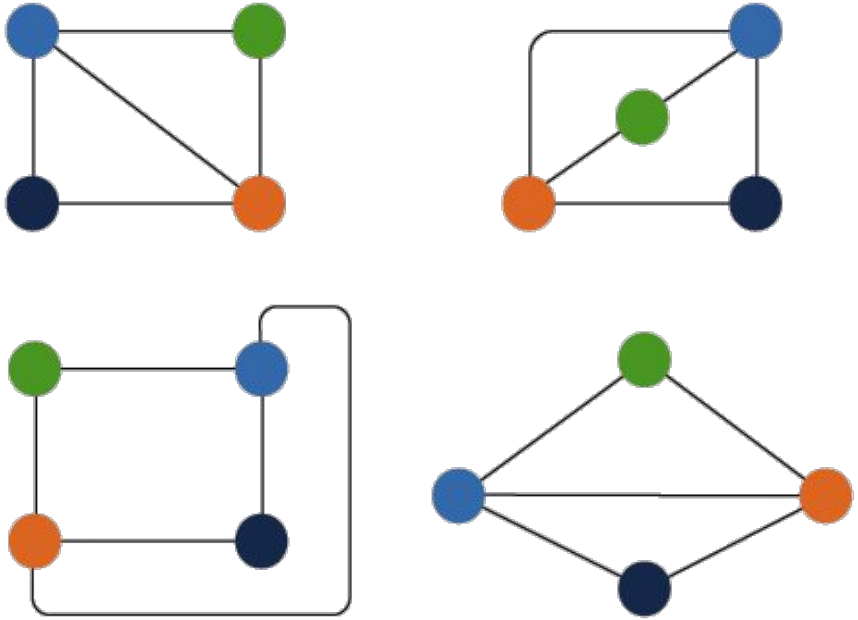
Problema del Caballo: Momento Filosófico (cont.)



Problema del Caballo: Código (v2.0)

```
def problema_del_caballo(n):
  # 1. Construir el grafo:
  #    - Crear los n x n vértices
  #    - Asignar las aristas a los vértices
  # 2. Buscar el camino hamiltoniano:
  return camino_hamiltoniano(grafo)
```

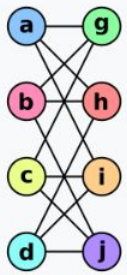
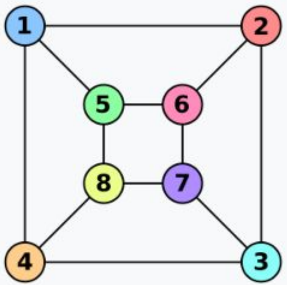
Isomorfismos de Grafos



www.javatpoint.com

Isomorfismos: Definición

Un isomorfismo de los grafos G y H es una biyección sobre los vértices de G y H f:V(G)→V(H) tal que cualquier par de vértices (u,v) son adyacentes en G sí y sólo si f(u) y f(v) en H también son adyacentes.

Graph G	Graph H	An isomorphism between G and H
		$f(a) = 1$ $f(b) = 6$ $f(c) = 8$ $f(d) = 3$ $f(g) = 5$ $f(h) = 2$ $f(i) = 4$ $f(j) = 7$

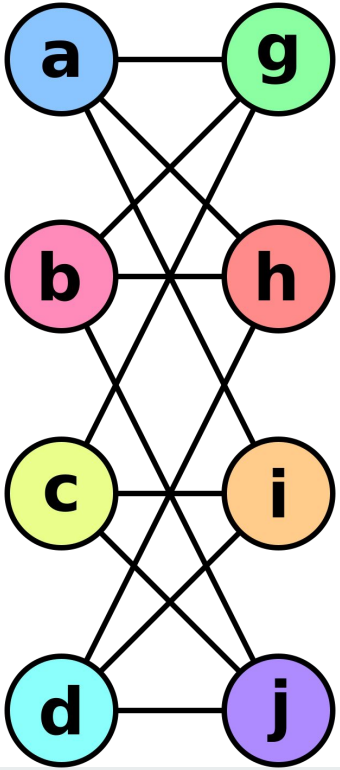
BooyaBazooka

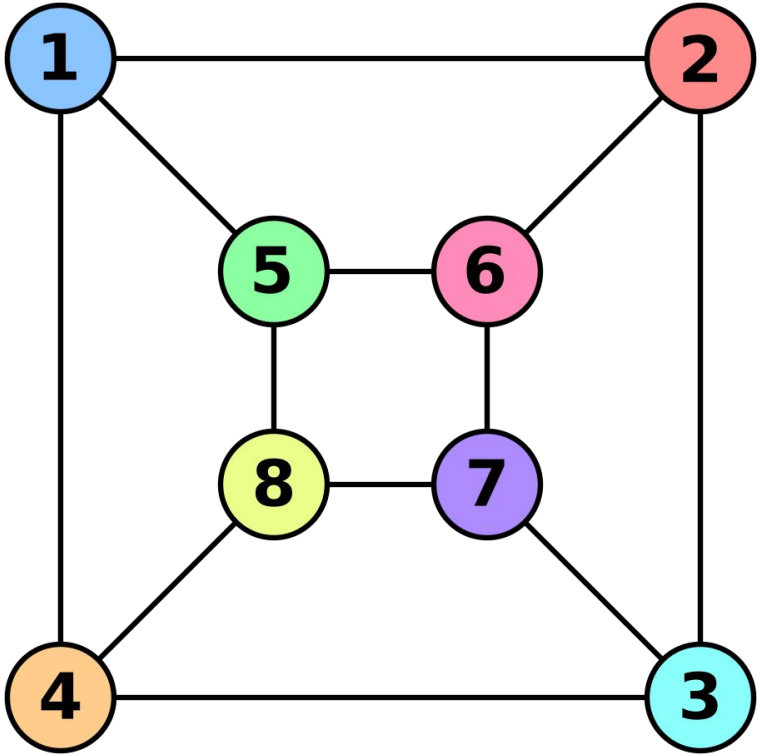
Isomorfismos: Análisis

¿Cómo podríamos saber si son isomorfos?

¿Cómo se resolvería por Fuerza Bruta?

¿Cómo se resolvería por Backtracking?





Isomorfismos: Análisis (cont.)

Por Fuerza Bruta:

- ¿{1,2,3,4,5,6,7,8} se corresponde con {a,b,c,d,g,h,i,j} ? No
- ¿{1,2,3,4,5,6,7,8} se corresponde con {a,b,c,d,g,h,j,i} ? No
- ¿{1,2,3,4,5,6,7,8} se corresponde con {a,b,c,d,g,i,j,h} ? No
- …
- Podríamos omitir 6! soluciones posibles sólo por detectar que cualquier solución que vincule {1,2} con {a,b} no puede ser válida

Isomorfismos: Podas posibles

¿Qué podas podemos aplicar?

- A nivel global:
  - Misma cantidad total de vértices en ambos grafos
  - Misma cantidad total de aristas en ambos grafos
- Para cada candidato parcial:
  - Descartar los vértices candidatos donde el grado es distinto
  - Descartar los vértices candidatos si las adyacencias son incompatibles
- Ciclos
  - Ambos grafos deben tener los mismos ciclos

Materias Compatibles



Yan Krukau

Materias Compatibles: Definición

Un estudiante debe decidir en qué cursos anotarse:

- Se tienen m Materias $\{M_1, M_2, ..., M_m\}$ para cursar
- Se conoce la lista de Cursos C_{ij} (con sus horarios) de cada Materia j (cada Materia j puede tener de 1 a n Cursos)
- No se puede anotar en más de un Curso por Materia
- No se puede anotar en dos Cursos que se superpongan
- Quiere cursar todas las Materias

Determinar todas las combinaciones de Materias y Cursos en que se puede anotar

Materias Compatibles: Análisis

- Las soluciones tendrán la forma $\{C_{i1}, C_{j2}, \ldots, C_{nm}\}$
- ¿Cuántas combinaciones pueden existir?
  - Si todas las m materias y cursos fueran compatibles entre sí, y si todas tuvieran n cursos disponibles, existirían n^m posibles combinaciones

¿Cómo se puede plantear algo por Backtracking?

Materias Compatibles: Estrategia

1. Si ya agoté todas las materias:
    a. Si el candidato parcial es factible
        i. devolver el candidato parcial
    b. Si no
        i. Devolver una solución vacía
2. Si el candidato parcial no es factible:
    a. Devolver una solución vacía
3. Tomar la próxima materia de la lista de materias y removerla
4. Para cada curso de la materia seleccionada:
    a. llamar recursivamente con el candidato parcial + curso seleccionado
    b. agregar las soluciones devueltas a la lista de soluciones halladas
5. Regresar la materia analizada a la lista de materias
6. Devolver la lista de soluciones halladas

Materias Compatibles: Estrategia (cont.)

Para determinar si un candidato parcial es factible:

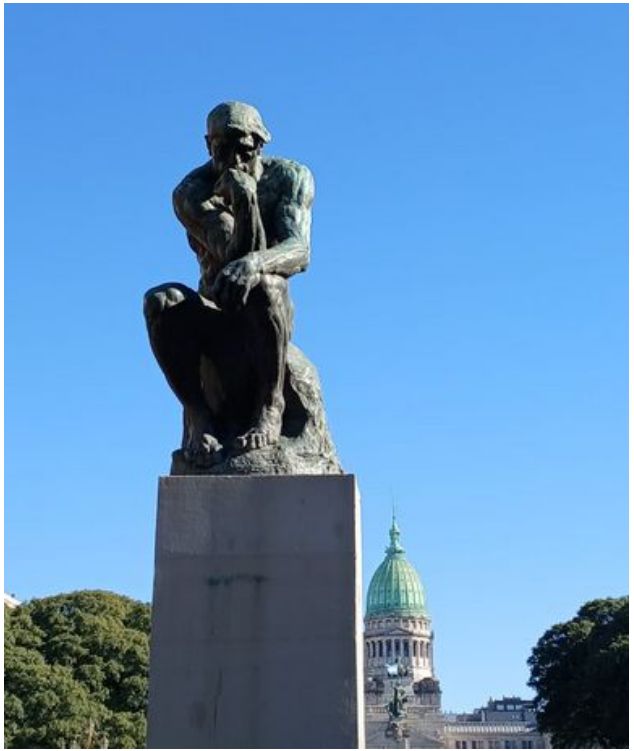
1. Se parte de un candidato parcial
2. Se toma el último curso agregado al candidato parcial
3. Para cada curso del candidato parcial
    a. Si el curso seleccionado es el último curso agregado:
        i. continuar con el próximo curso del candidato parcial
    b. Si el curso seleccionado y el último curso agregado no son compatibles
        i. terminar y devolver False
4. Llegados hasta aquí, el candidato parcial y el último curso agregado son compatibles: devolver True

Materias Compatibles: Momento Filosófico

¿Se puede establecer una analogía entre este problema y alguno de grafos que ya conozcamos?

¿Qué serían los vértices?

¿Qué serían las aristas?

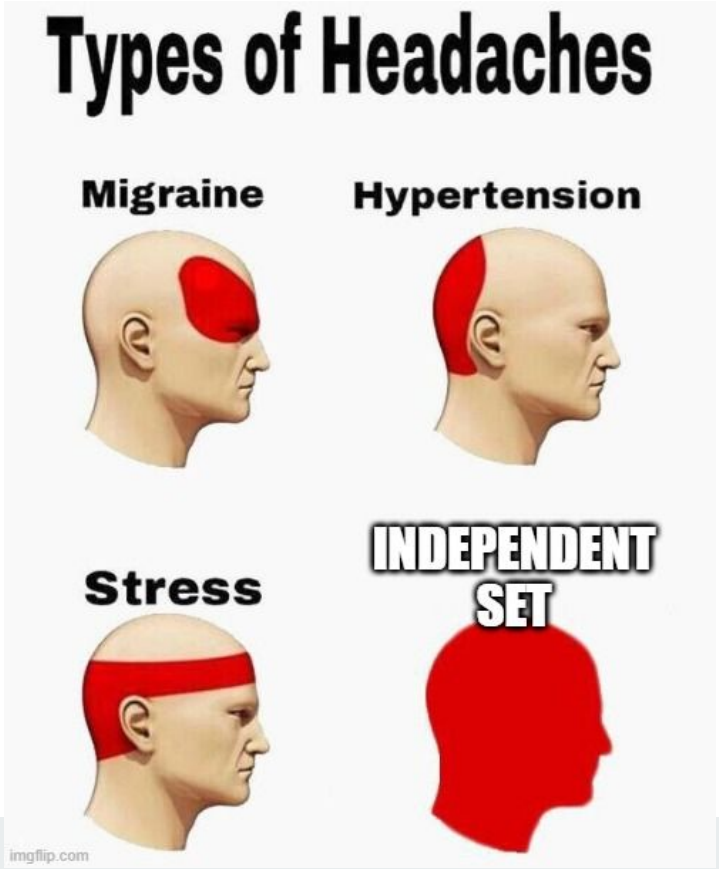


Materias Compatibles: Momento Filosófico

- Los vértices podrían ser los cursos C_{ij}
- Las aristas unirían los cursos de materias distintas que son incompatibles entre sí
- Las aristas también unirían todos los cursos de una misma materia entre sí

¿Se parece a algo que hayamos visto?

Materias Compatibles: Momento Filosófico



Sumatoria de N Dados



Pixabay

Sumatoria de N Dados: Definición

- Se tienen N dados de M caras
- Se desea sumar un total S

Determinar todos los lanzamientos de los N dados que suman S

Sumatoria de N Dados: Análisis

- Las soluciones tendrán la forma $\{D_1, D_2, ..., D_n\}$, donde D_i es el valor que dio el $i\text{-}ésimo$ dado al ser lanzado
- ¿Cuántas combinaciones pueden existir?
  - Si m es la cantidad de caras del dado, existirían m^n posibles permutaciones

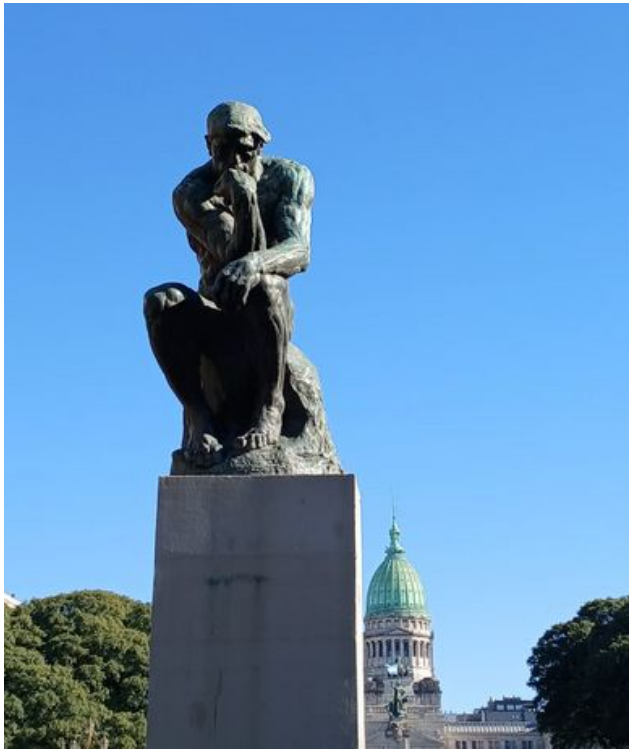
¿Cómo se puede plantear algo por Backtracking?

Sumatoria de N Dados: Estrategia

1. Si la cantidad de elementos del candidato parcial es N y su suma es S:
   a. Agregar el candidato parcial a la lista de soluciones y volver
2. Podas:
   a. Si la longitud del candidato parcial es N, volver
   b. Si Sum(candidato parcial) es mayor o igual que S, volver
   c. Si Sum(candidato parcial) + [N-Len(candidato parcial)]*M < S, volver
   d. Si Sum(candidato parcial) + [N-Len(candidato parcial)]*1 > S, volver
3. Para cada valor posible de un lanzamiento:
   a. Incrementar el candidato parcial con el valor
   b. Llamar recursivamente con el candidato parcial + valor obtenido
   c. Descartar el candidato parcial incrementado
4. Volver

Sumatoria de N Dados: Momento Filosófico

¿Se puede establecer una analogía entre este problema y alguno ~~de grafos~~ que ya conozcamos?

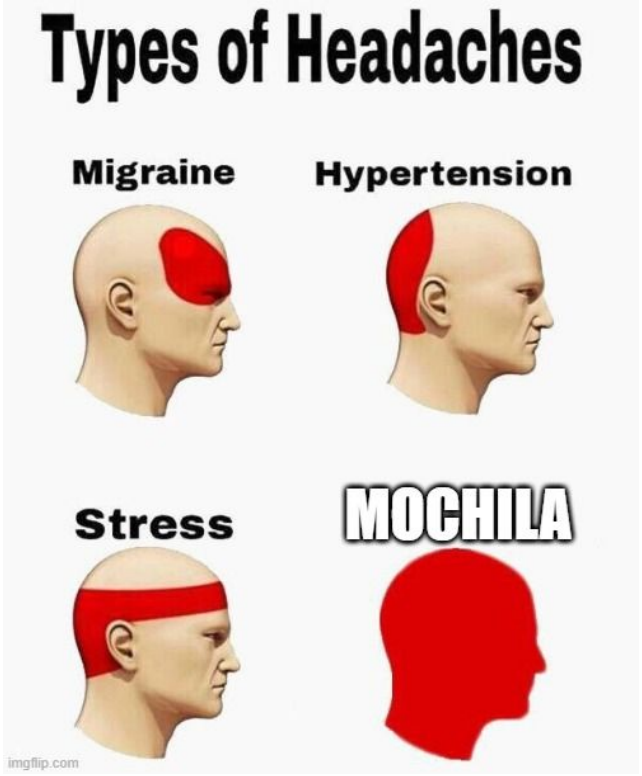


Sumatoria de N Dados: Momento Filosófico (cont.)

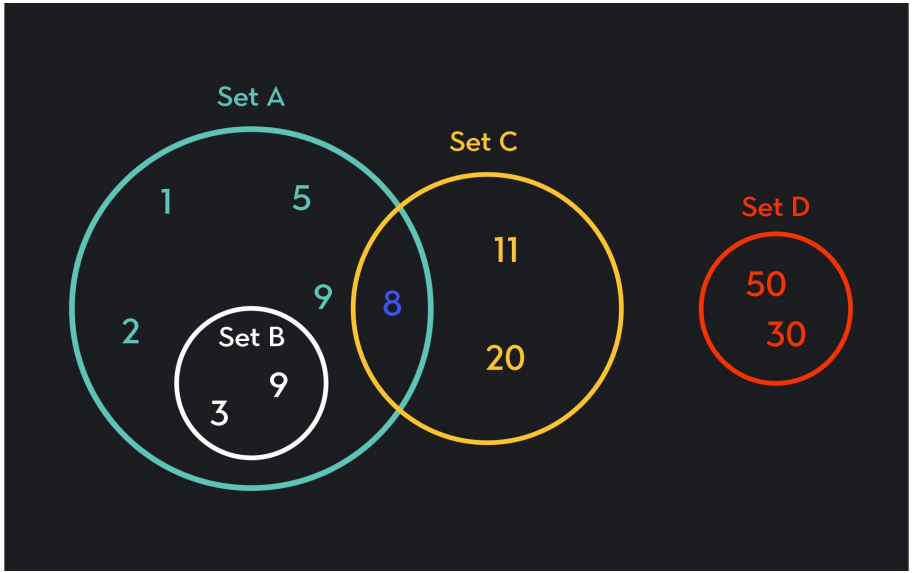
- Cada lanzamiento posible podría ser un elemento a considerar
- El valor de cada dado podría ser su peso
- La suma total de los dados podría ser la capacidad a completar

¿Se parece a algo que hayamos visto?

Materias Compatibles: Momento Filosófico (cont.)



Suma de Subconjuntos (Subset Sum)



Sarah Thomas

Suma de Subconjuntos: Definición

- Se tiene un *multiset*(*) S de n números naturales
- Se define un valor T como suma objetivo

Se quiere determinar si existe un subconjunto de S tal que sus elementos sumen exactamente T

(*) Un *multiset* es un *set* que admite elementos repetidos

Suma de Subconjuntos: Análisis

- Las soluciones tendrán la forma $\{e_1, e_2, \ldots, e_k\}$, donde e_i es un elemento de S
- ¿Cuántas combinaciones pueden existir?
  - Si hay n elementos en S, habría 2^n subconjuntos
  - Verificar la suma de cada subconjunto demandaría $O(n)$
  - La complejidad de Fuerza Bruta sería $O(2^n n)$

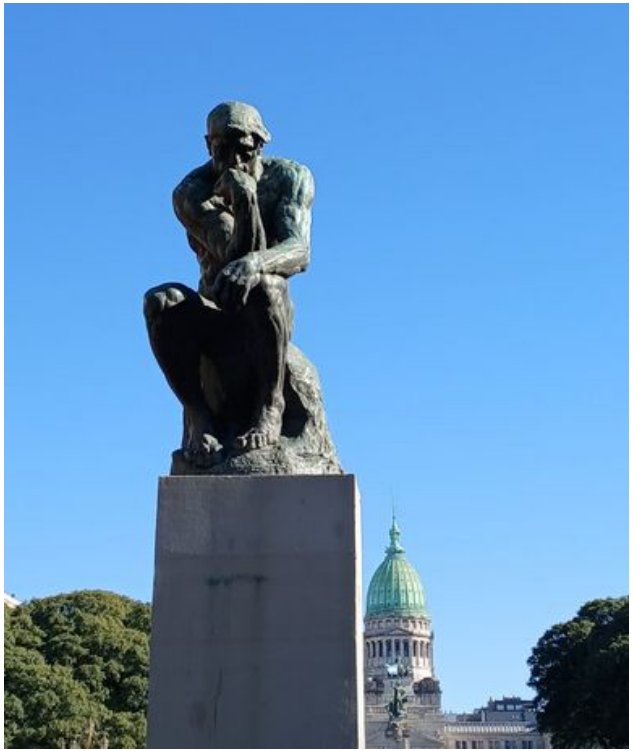
¿Cómo se puede plantear algo por Backtracking?

Suma de Subconjuntos: Estrategia

1. Si el candidato parcial suma T, devolverlo
2. Podas:
    a. Si la suma del candidato parcial excede T, devolver una solución vacía
    b. Si no quedan elementos por analizar, devolver una solución vacía
3. Incrementar el candidato parcial con el próximo elemento de S (pero con un elemento menos en S)
4. Llamar recursivamente con el candidato parcial incrementado
5. Si la llamada recursiva devuelve una solución:
    a. Devolverla
6. Llamar recursivamente con el candidato sin incrementar (pero con un elemento menos en S)
7. Devolver el resultado de la llamada recursiva

Suma de Subconjuntos: Momento Filosófico

¿Se puede establecer una analogía entre este problema y alguno ~~de grafos~~ que ya conozcamos?



Suma de Subconjuntos: Momento Filosófico (cont.)



Podas



Jimmy Chan

Podas en Problemas de Optimización

1. Tengo un Problema de Optimización (maximizar o minimizar xxxxx)
2. Ya encontré una solución completa y válida, conozco su valor
3. Puedo usarla como cota para podar:
    - En un problema de minimización, usar ese valor conocido para podar candidatos parciales que lo excedan
    - En un problema de maximización, usar ese valor conocido para podar un candidato parcial que nunca logre alcanzarlo

Podas en Problemas de Optimización (cont.)

¿Cómo puedo obtener una cota desde la primera iteración para mejorar mis podas?

1. Usar un algoritmo greedy para encontrar una primera solución
2. Usar ese resultado (subóptimo) como cota para las primeras podas
3. Cuando encuentro una solución por Backtracking, actualizo la cota y sigo podando



Próxima clase

8/9/2026	**Práctica D&C, Greedy y Backtracking.**	
15/9/2026	**Programación Dinámica II**	

D y C:	Ejercicio #12 (Buchwald)
Greedy:	Ejercicio #16 (Buchwald)
Backtracking	Ejercicio #8 (Buchwald)



¡Muchas gracias!